Buổi 1

Kiến trúc dự án lớn & khởi tạo dự án mẫu

Angular 22 · Nx 24 · ng-zorro · Spring Boot 3.5

Mục tiêu buổi học

Sau buổi này, học viên có thể:

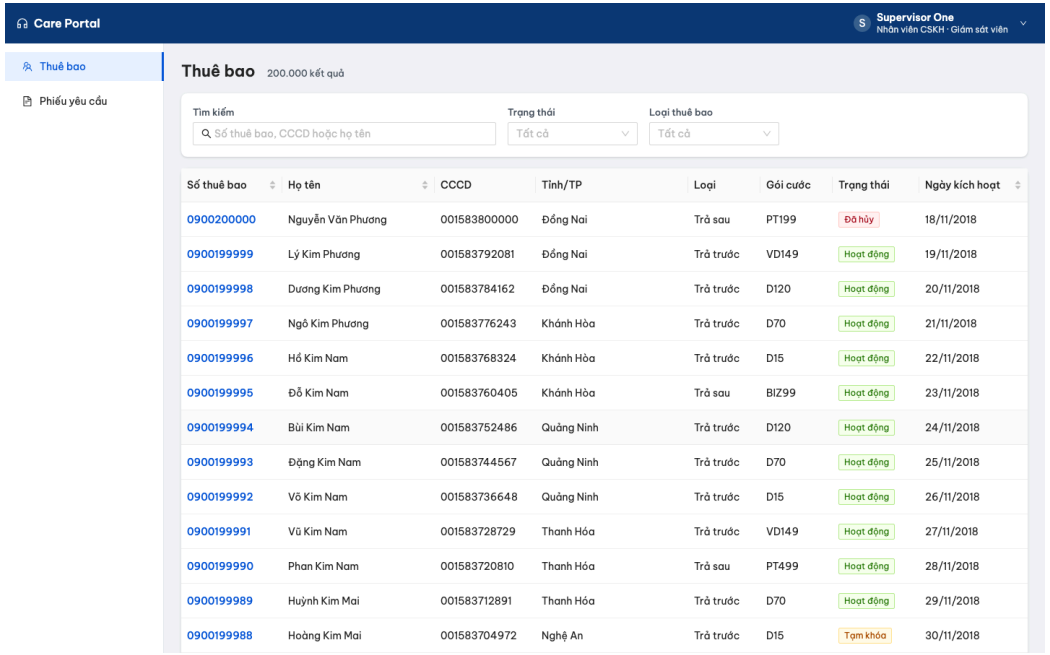
1. Giải thích **vì sao** dự án lớn cần tổ chức lại code, và khi nào nên dùng monorepo
2. Đọc và mở rộng một **Nx workspace**: apps, libs, tags, đồ thị phụ thuộc
3. Áp dụng **4 loại lib** (feature / ui / data-access / util) và **module boundaries**
4. Dùng **lazy loading** để tách bundle theo nghiệp vụ
5. Dùng **runtime config**: build một lần, deploy nhiều môi trường

Lab: thêm màn hình "Gói cước" đúng kiến trúc, phá luật phụ thuộc rồi sửa, làm nhãn môi trường

Dự án mẫu xuyên suốt khóa học: Care Portal

Hệ thống CSKH & quản lý thuê bao viễn thông

- **portal**: nhân viên CSKH tra cứu thuê bao, phiếu yêu cầu
- **admin**: quản trị, thống kê
- 200.000 thuê bao, 50.000 phiếu (có thể tăng lên hàng triệu)
- 3 vai trò: AGENT, SUPERVISOR, ADMIN
- Đăng nhập bằng JWT; buổi 5 chuyển sang SSO (Keycloak)



Kiến trúc tổng thể



Học viên chỉ cần **Node + Docker**. Mỗi buổi học sẽ đào sâu một phần trong sơ đồ này.

Dự án "nhỏ" trở thành "lớn" khi nào?

Dấu hiệu	Hệ quả nếu không có kiến trúc
Nhiều team cùng sửa một repo	Xung đột code, sửa chỗ này hỏng chỗ khác
Nhiều app dùng chung logic (auth, API, UI)	Copy-paste, mỗi app một phiên bản
Hàng trăm màn hình	Bundle phình to, tải trang chậm
CI chạy toàn bộ test mỗi lần commit	Pipeline 30–60 phút
Nhiều môi trường (DEV/TEST/UAT/PROD)	Mỗi môi trường build riêng → "chạy ở TEST, lỗi ở PROD"

> Kiến trúc là đặt **ranh giới** để code tăng thêm mà độ phức tạp không tăng theo cấp số nhân.

Monorepo hay Polyrepo?

Polyrepo: mỗi app/lib một repo

- ✅ Độc lập hoàn toàn, phân quyền dễ
- ❌ Dùng chung code phải publish npm
- ❌ Đổi API → sửa N repo, N lần release
- ❌ Lệch phiên bản Angular giữa các repo

Monorepo: nhiều project, một repo

- ✅ Đổi code dùng chung + các nơi dùng trong **1 commit**
- ✅ Một phiên bản Angular/thư viện
- ✅ Refactor toàn cục an toàn
- ❌ Cần công cụ: chỉ build/test phần bị ảnh hưởng, chặn import lung tung

> Monorepo không có nghĩa là một khối code lớn: bên trong vẫn chia module với ranh giới rõ ràng. Công cụ để làm việc đó là **Nx**.

Nx mang lại gì?

Tính năng	Ý nghĩa
Project graph	Biết project nào phụ thuộc project nào (phân tích import)
Affected	`nx affected -t test` : chỉ chạy phần bị ảnh hưởng bởi thay đổi
Cache	Kết quả build/test được cache, chạy lại không đổi thì trả kết quả ngay
Generators	Tạo app/lib/component theo đúng quy ước của team
Module boundaries	Luật ESLint cấm import sai tầng

```
npx nx graph                          # xem đồ thị
npx nx run-many -t lint test build # chạy cho mọi project
npx nx affected -t test               # chỉ project bị ảnh hưởng (buổi 7)
```

Cấu trúc workspace Care Portal

```
frontend/
  apps/
    portal/                      ← "vỏ": routing, cấu hình, ghép các feature
    admin/
    portal-e2e/
  libs/
    shared/util-config           type:util          cấu hình runtime
    shared/data-access-auth      type:data-access   AuthService, interceptor, guard
    shared/feature-auth          type:feature       trang đăng nhập
    shared/data-access-api       type:data-access   API client, model
    shared/ui-layout             type:ui            Shell, trang 403/404
    portal/feature-subscribers   type:feature       màn hình thuê bao
    portal/feature-tickets       type:feature       màn hình phiếu
    admin/feature-dashboard      type:feature       thống kê
```

> App nên **mỏng**: gần như chỉ có routing và provider. Code nghiệp vụ nằm trong lib.

Đồ thị phụ thuộc (`nx graph`)

Tầng	Project	Phụ thuộc vào
app	`portal`	feature-subscribers, feature-tickets, feature-auth, data-access-auth, ui-layout, util-config
app	`admin`	feature-dashboard, feature-auth, data-access-auth, ui-layout, util-config
feature	`portal-feature-subscribers`	data-access-api, data-access-auth
feature	`portal-feature-tickets`	data-access-api, data-access-auth
feature	`admin-feature-dashboard`	data-access-api
feature	`shared-feature-auth`	data-access-api, data-access-auth
data-access	`shared-data-access-api` / `-auth`	util-config
ui	`shared-ui-layout`	*(không phụ thuộc lib nào)*
util	`shared-util-config`	*(không phụ thuộc lib nào)*

4 loại lib

Loại	Chứa gì	Ví dụ trong dự án
feature	Màn hình, smart component, route của một nghiệp vụ	`feature-subscribers`
ui	Component trình bày, chỉ có `input()` / `output()`, không gọi API	`ui-layout` (Shell)
data-access	Service gọi API, state, interceptor	`data-access-api`, `data-access-auth`
util	Hàm thuần, type, hằng số, không phụ thuộc Angular	`util-config`

Chiều phụ thuộc **chỉ đi xuống**:

```
app ⟶ feature ⟶ ui / data-access ⟶ util
```

Tags: hai chiều phân loại

Mỗi lib mang 2 tag trong `project.json`:

```
{ "name": "portal-feature-subscribers",
  "tags": ["scope:portal", "type:feature"] }
```

- `type:*` → **tầng kiến trúc** (feature / ui / data-access / util)
- `scope:*` → **miền nghiệp vụ / app** (portal / admin / shared)

Quy ước đặt tên: `<scope>-<type>-<tên>` → import `@care/portal-feature-subscribers`

Module boundaries: luật được máy kiểm tra

`eslint.config.mjs` (thư mục gốc):

```
depConstraints: [
  { sourceTag: 'type:app',     onlyDependOnLibsWithTags: ['type:feature','type:ui','type:data-access','type:util'] },
  { sourceTag: 'type:feature', onlyDependOnLibsWithTags: ['type:ui','type:data-access','type:util'] },
  { sourceTag: 'type:ui',      onlyDependOnLibsWithTags: ['type:ui','type:util'] },
  { sourceTag: 'type:data-access', onlyDependOnLibsWithTags: ['type:data-access','type:util'] },
  { sourceTag: 'type:util',    onlyDependOnLibsWithTags: ['type:util'] },
  { sourceTag: 'scope:portal', onlyDependOnLibsWithTags: ['scope:portal','scope:shared'] },
  { sourceTag: 'scope:admin',  onlyDependOnLibsWithTags: ['scope:admin','scope:shared'] },
  { sourceTag: 'scope:shared', onlyDependOnLibsWithTags: ['scope:shared'] },
]
```

> Code review bằng mắt sẽ có lúc sót, còn lint thì chạy ở mọi commit. Vi phạm luật thì CI báo đỏ.

Vi phạm thì sao? (lỗi thật từ dự án)

Feature import feature khác:

```
error  A project tagged with "type:feature" can only depend on libs tagged with
       "type:ui", "type:data-access", "type:util"   @nx/enforce-module-boundaries
```

Đi vòng qua public API bằng đường dẫn tương đối:

```
import { SubscriberListPage } from '../../../feature-subscribers/src/lib/...';
```

```
error  Projects cannot be imported by a relative or absolute path,
       and must begin with a npm scope              @nx/enforce-module-boundaries
```

> `index.ts` là **public API** của lib. Những gì không export ra ở đó là chi tiết nội bộ, lib được phép đổi mà không cần báo ai.

Lazy loading theo nghiệp vụ

`apps/portal/src/app/app.routes.ts`

```
export const appRoutes: Route[] = [
  { path: 'login', canActivate: [guestGuard],
    loadComponent: () => import('@care/shared-feature-auth').then((m) => m.LoginPage) },
  {
    path: '',
    canActivateChild: [authGuard],          // chưa đăng nhập → /login?returnUrl=...
    children: [
      { path: '', pathMatch: 'full', redirectTo: 'subscribers' },
      { path: 'subscribers',
        loadChildren: () => import('@care/portal-feature-subscribers').then((m) => m.subscriberRoutes) },
      { path: 'tickets',
        loadChildren: () => import('@care/portal-feature-tickets').then((m) => m.ticketRoutes) },
      { path: 'forbidden', component: ForbiddenPage },
      { path: '**', component: NotFoundPage },
    ],
  },
];
```

Bằng chứng: output của `nx build portal`

```
Initial chunk files | Names                  |  Raw size | Estimated transfer size
styles-MURJOT2J.css | styles                 | 576.99 kB |               52.94 kB
                    | Initial total          |   1.29 MB |              233.45 kB

Lazy chunk files    | Names                  |  Raw size | Estimated transfer size
chunk-C6JmX3Gg.js   | subscriber-detail-page |  41.00 kB |               11.05 kB
chunk-slCUYiXN.js   | subscriber-list-page   |   7.23 kB |                2.69 kB
chunk-B8En6oCa.js   | ticket-list-page       |   7.09 kB |                2.59 kB
```

- Mỗi màn hình là **một chunk riêng**, chỉ tải khi người dùng vào màn hình đó
- Thêm 100 màn hình thì bundle ban đầu **không tăng**

> Initial bundle 1,29 MB, trong đó riêng CSS (toàn bộ ng-zorro) đã 577 kB → bài tập tối ưu ở **buổi 3**.

Cấu hình hiện đại: function thay cho NgModule

`apps/portal/src/app/app.config.ts`

```
export function appConfig(config: AppConfig): ApplicationConfig {
  return {
    providers: [
      provideBrowserGlobalErrorListeners(),
      provideAppConfig(config),
      provideHttpClient(withInterceptors([authInterceptor])),
      provideRouter(appRoutes, withComponentInputBinding()),
      provideAuth(),        // ← lib data-access-auth tự đóng gói
      provideUi(),          // ← lib ui-layout: i18n vi_VN, icon
    ],
  };
}
```

- Mỗi lib cung cấp một hàm `provideXxx()` → app chỉ việc **ghép** lại
- Standalone component, **zoneless**, signals là mặc định từ Angular 20+

Runtime config: build một lần, deploy nhiều nơi

❌ `environment.ts`

- Giá trị đóng cứng vào bundle lúc build
- DEV/TEST/PROD = 3 bản build khác nhau
- Bản đã test ≠ bản lên PROD

✅ `config.json` **đọc lúc khởi động**

- 1 bản build, đổi file config theo môi trường
- Đúng tinh thần "build once, deploy many"
- DevOps đổi URL mà không cần dev

```
// apps/portal/src/main.ts
loadAppConfig()                                    // fetch('/config.json')
  .then((config) => bootstrapApplication(App, appConfig(config)))
  .catch((err) => console.error(err));
```

Runtime config trong code

`libs/shared/util-config/src/lib/app-config.ts`

```ts
export type AuthConfig =
  | { mode: 'local' }                                    // JWT do backend phát
  | { mode: 'oidc'; issuer: string; clientId: string };  // Keycloak (buổi 5)

export interface AppConfig {
  apiUrl: string;
  auth: AuthConfig;
}

export const APP_CONFIG = new InjectionToken<AppConfig>('APP_CONFIG');
```

Bất cứ đâu cần thì inject:

```ts
private readonly baseUrl = `${inject(APP_CONFIG).apiUrl}/api/subscribers`;
```

`apps/portal/public/config.json` → copy nguyên vào `dist/` khi build

Generators: quy ước viết một lần, dùng mãi

`nx.json`

```
"generators": {
  "@nx/angular:library": {
    "linter": "eslint", "unitTestRunner": "vitest-analog",
    "standalone": true, "skipModule": true, "prefix": "care", "style": "scss"
  },
  "@nx/angular:component": { "style": "scss", "changeDetection": "OnPush" }
}
```

```
npx nx g @nx/angular:library libs/portal/feature-service-plans \
  --name=portal-feature-service-plans --tags=scope:portal,type:feature
```

> Thiếu `--name` → project tên `feature-service-plans`, import `@care/feature-service-plans`, lệch quy ước.

Quyết định kiến trúc: ghi lại bằng ADR

ADR (Architecture Decision Record): vài đoạn văn trả lời *bối cảnh, quyết định, hệ quả*.

Ví dụ thật từ dự án này: **chọn thư viện UI**

	PrimeNG 22	ng-zorro-antd 22
License	Thương mại (PrimeUI): miễn phí khi doanh thu < 1 triệu USD, < 5 dev	MIT
Hỗ trợ Angular 22	Có	Có
Bảng/form doanh nghiệp	Mạnh	Mạnh

→ Quyết định: **ng-zorro-antd**. Tổ chức lớn mà dùng PrimeNG thì phải mua license theo số dev.

> Trước khi chọn thư viện, luôn kiểm tra **license**, không chỉ kiểm tra tính năng.

Micro-frontend thì sao?

Cần micro-frontend (Module Federation) khi	Chưa cần khi
Nhiều team **deploy độc lập**, lịch release khác nhau	Một team / vài team cùng nhịp release
Ghép app của nhiều đơn vị, công nghệ khác nhau	Cùng Angular, cùng repo
Cần cập nhật một phần mà không deploy lại toàn bộ	Lazy loading đã đủ nhanh

Cái giá phải trả: phức tạp về phiên bản thư viện dùng chung, routing, auth, debug.

> Bắt đầu bằng **monorepo + lib có ranh giới rõ**. Nếu ranh giới đã đúng thì sau này tách sang micro-frontend chỉ là đổi cách deploy.

Phía backend: cùng tư duy ranh giới

```
backend/src/main/java/com/careportal/api/
  config/       SecurityConfig, CORS, OpenAPI, cache
  common/       PageResponse, ProblemDetail (lỗi chuẩn RFC 9457)
  subscriber/   Entity, Repository, Service, Controller, DTO, Mapper
  ticket/
  plan/
```

Hợp đồng ổn định giữa FE và BE:

```
// Phân trang
{ "content": [...], "page": 0, "size": 20, "totalElements": 200000, "totalPages": 10000 }
// Lỗi
{ "status": 409, "detail": "So thue bao 0812345678 da ton tai" }
```

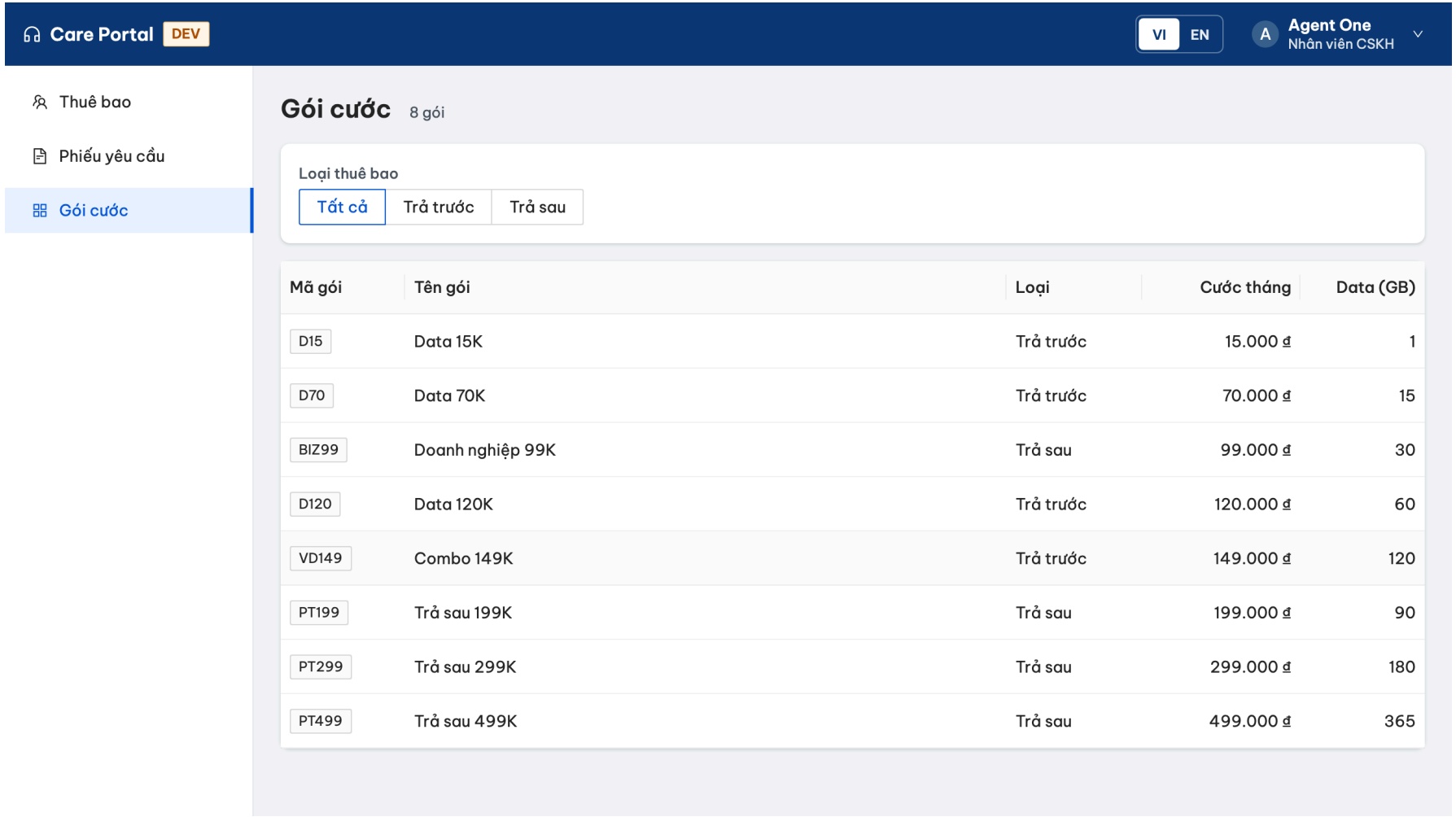
Đóng gói theo **nghiệp vụ** (package-by-feature) thay vì chia theo controller/service/repository.

Lab (90 phút)

Bài	Nội dung	Thời gian
1	Khám phá workspace: `nx graph`, tags, lazy chunk	15'
2	Thêm màn hình **Gói cước**: generator → page → route → menu	45'
3	Phá luật phụ thuộc, đọc lỗi, thảo luận cách tách lib	15'
4	Nhãn môi trường từ `config.json`, đổi mà không build lại	15'

Đề bài: `course/session-01/lab/README.md`

Kết quả mong đợi của Bài 2 + 4



Bẫy hay gặp khi làm lab

```
// ❌ Bảng trống, KHÔNG có request nào
plans = rxResource({
  params: () => this.type() ?? undefined,     // undefined → resource "idle"
  stream: ({ params }) => this.api.list(params),
});

// ✅
plans = rxResource({
  params: () => ({ type: this.type() }),
  stream: ({ params }) => this.api.list(params.type ?? undefined),
});
```

Thêm 2 lưu ý:

- Đọc `value()` khi resource đang lỗi → **throw**. Luôn kiểm tra `hasValue()` trước.
- Dùng icon mới → phải đăng ký trong `provideUi()`, nếu không icon sẽ không hiện.

Tổng kết

1. Dự án lớn cần **ranh giới**: apps mỏng, logic nằm trong libs
2. **4 loại lib × scope**: tag hóa và để ESLint kiểm tra
3. **Public API** (`index.ts`): không import xuyên vào file nội bộ của lib khác
4. **Lazy loading** theo nghiệp vụ, kiểm chứng bằng output build
5. **Runtime config**: một bản build cho mọi môi trường
6. Quyết định kiến trúc → ghi lại bằng **ADR** (kể cả license)

Buổi 2: State management: Signals, RxJS, NgRx. Khi nào dùng cái nào?